Bags, Vectors & Transformers

Anonymizing Text · Bonus, Part 4

Denise J. Roth, PhD Candidate
Strategic Communication Group · Wageningen University & Research

What Counts as Identifying?

Direct identifiers

point to one person on their own: name, e-mail, phone number, home address, bank account, citizen service number (BSN)

Indirect identifiers

harmless alone, identifying together: "a dairy farmer in her fifties, the only woman on the village council, daughter studying in Utrecht"

	what it means	GDPR?
pseudonymized	codes (`PERSON_1`); a key exists somewhere	applies
anonymized	no reasonable way left to identify anyone	doesn't

Most "anonymized" research data is, strictly, **pseudonymized**. Fine, as long as you treat it that way. Truly anonymizing rich text is hard, and never achieved by software alone.

Two Kinds of Detectors

Rules: regular expressions

For identifiers with a fixed *format*: e-mail, phone, postcode, IBAN.

```
"POSTCODE":  r"\b[1-9]\d{3}\s?[A-Z]{2}\b"
"IBAN":  r"\bNL\d{2}\s?[A-Z]{4}(?:\s?\d{2,4}){3}\b"
```

Transparent and fast. But "**2019 EU**" and "**1450 HF**" (cows) match the postcode pattern too.

Models: named entity recognition

For names, places, organizations. spaCy, small English model, on the notebook's test transcripts:

text	tagged as
Femke de Vries	ORG
Kerkweg (a street)	ORG
Marieke	PERSON
marieke (lowercase)	missed
Utrecht	GPE, but not identifying

A wrong type is a small problem (still masked); a miss is a big one.

Measuring What You Caught

The notebook generates transcripts from templates, so it knows where every identifier is.

Recall: of all identifiers, how many were masked? A miss is a **leak**.

Precision: of everything masked, how much was an identifier? A false alarm costs **information**, but leaks nothing.

Count an identifier as caught only if it's *fully* masked: hiding "Pieter" but not "de Vries" still leaks.

English transcripts, small spaCy model

detector	recall	precision
regular expressions only	0.22	1.00
NER only	0.60	0.85
regex + NER	0.80	0.88
+ date pattern	0.85	0.88

Per type: e-mail, phone, postcode, IBAN 1.00; names 0.78; addresses only 0.38 fully masked (the street, not the number).

One run of the notebook; your numbers depend on the spaCy version. Real data is messier than templates.

Two Lessons from the Exercises

Measure on your own material

The small *Dutch* model on the Dutch transcripts: recall 0.63, precision 0.64. Worse than English.

The biggest false alarm: the speaker label `I:` tagged as a **person**, about a hundred times. Ignoring speaker labels lifts precision to 0.87.

A tool's reputation says little about your language and text type.

Find once, mask everywhere

The model misses "*Joost* says…" and a lowercase "*marieke*". But if it found "Joost Mulder" or "Marieke" elsewhere in the document, search for all other occurrences, ignoring case.

Name recall: 0.78 → 0.86, precision unchanged.

Replacing: Three Strategies

strategy	example	pros	cons
redaction	`[PERSON]`	simple, clearly marked	who is who?
consistent pseudonyms	`[PERSON_1]`, `[PERSON_2]`	keeps who is who, across the corpus	needs a key
surrogates	Jan Jansen → Kees Mulder	reads naturally	a missed real name hides among fake ones; wrong type gives nonsense

```
R: My name is [PERSON_3].  I farm near [LOCATION_2].
R: My neighbor [PERSON_4] sold his cows.
R: [PERSON_4] and I went to the protest together.
```

The mapping from codes to names **is the key**: whoever has it can undo everything. Store it apart from the data, encrypted, or delete it.

What No Tool Catches

```
R: My wife is the only veterinarian in [LOCATION_4] who still visits farms at night.
```

Nothing here is a name, yet with the other details in the interview it may point to one family. Neighbors know far more than an outside attacker.

What helps

- **generalize**: "a village in Gelderland", "in her fifties"
- read with the question: *could someone from the area recognize them?*
- share less: coded data, or access on request

Let software prepare the human check. List capitalized words that survived anonymization and aren't sentence-initial. In the notebook, most hits are real leaks: missed streets, villages, names, and month names from dates of birth.

Lowercase names slip through this check too, so also read a random sample in full.

Tools and a Workflow That Holds Up

Tools worth knowing

- **Presidio** (Microsoft): patterns + NER + context words, extensible
- **deduce**: Dutch (medical) text, UMC Utrecht
- **Textwash**: open source, tested against re-identification
- **larger NER models**: `en_core_web_trf`, `nl_core_news_lg`, or a fine-tuned BERT (Day 3)

LLMs spot indirect identifiers well, but sending confidential transcripts to an online API is the very transfer you wanted to avoid. Use a local model.

Workflow

1. Decide what must go (DMP, consent forms)
2. Patterns + NER, consistent pseudonyms; key stored separately
3. Measure recall per type on a hand-checked sample of *your* data
4. Check a list of leak candidates; read a random sample
5. Handle indirect identifiers by hand
6. Document what you did, so others can judge the remaining risk

In the Notebook

- Generating English and Dutch transcripts with known identifiers
- Regular expressions for e-mail, phone numbers, postcodes, IBANs and dates
- spaCy named entity recognition, and combining it with rules
- Measuring recall and precision per identifier type, in two languages
- Redaction, consistent pseudonyms with a key, and surrogates
- Finding names everywhere once they're found once, and listing leak candidates

`bonus/notebooks/4_anonymizing_text_exercises.ipynb` · 5 exercises · no GPU needed